

Шпаргалка по безопасности Docker

Защита образов, контейнеров и узла: сборка без лишнего, запуск без root, ограничение возможностей и ресурсов, секреты, сеть и реестр, проверка конфигурации, признаки неблагополучия.

Что важно понимать

- Контейнеры делят ядро с узлом: это изоляция процессов, а не виртуальная машина.
- Доступ к сокету `/var/run/docker.sock` равносителен правам root на узле.
- Членство в группе `docker` — тоже фактически права root.
- Слои образа хранят историю: удалённый в следующем слое секрет остаётся в предыдущем.
- Тег `latest` не даёт воспроизводимости: сегодня и завтра это разные образы.

Образ

`FROM node:20-alpine / distroless`

Минимальная база: меньше пакетов — меньше уязвимостей

Многоэтапная сборка

Инструменты сборки остаются в промежуточном слое, в итоговый образ не попадают

Фиксация версии

Конкретный тег или цифровой отпечаток: `FROM alpine@sha256:...`

`USER app`

Не запускать приложение от root — создайте пользователя в образе

`.dockerignore`

Не отправлять в контекст сборки `.git`, `.env`, ключи, каталоги зависимостей

`--build-arg` для секретов

Так делать не надо: значения видны в истории. Используйте `RUN --mount=type=secret`

`docker history образ`

Посмотреть, что попало в слои

Запуск контейнера

`--user 1000:1000`

Явный непривилегированный пользователь

`--read-only`

Файловая система только для чтения; изменяемое — в тома или `--tmpfs`

`--cap-drop=ALL --cap-add=NET_BIND_SERVICE`

Убрать все возможности ядра и вернуть только нужную

`--security-opt no-new-privileges`

Запретить повышение прав внутри контейнера

`--pids-limit 200`

Защита от исчерпания процессов

`--memory 512m --cpus 1`

Ограничение ресурсов: один контейнер не уронит узел

`--network none`

Полная сетевая изоляция для задач, которым сеть не нужна

`-p 127.0.0.1:8080:8080`

Публиковать порт только локально, а не на все интерфейсы

Чего избегать: `--privileged`, `--cap-add=SYS_ADMIN`, монтирования `/var/run/docker.sock` и каталогов узла вроде `/` или `/etc` внутрь контейнера.

Секреты

- Не класть в образ и не передавать через ARG: они остаются в слоях и в истории сборки.
- Переменные окружения видны в `docker inspect` и в журналах — приемлемо не для всего.
- Надёжнее: секреты Docker Swarm и Kubernetes, монтирование файлом, внешние хранилища.
- В сборке: `RUN --mount=type=secret,id=npmrnc ...` — значение не попадает в слой.
- Файлы с секретами монтируйте только для чтения и с правами, закрытыми для остальных.

Проверка образов

`docker scout cves образ`

Известные уязвимости в компонентах образа

`trivy image образ`

Популярный сканер: пакеты, секреты, ошибки конфигурации

`grype образ / syft образ`

Поиск уязвимостей и построение перечня компонентов

`hadolint Dockerfile`

Линтер: находит небезопасные и неэффективные конструкции

`cosign sign / cosign verify`

Подпись образов и проверка подлинности перед выкладкой

Сканирование полезно в конвейере на каждый образ и повторно по расписанию: новые уязвимости находят в уже собранных образах.

Узел и демон

Обновления

Держите Docker и ядро в актуальном состоянии: изоляция опирается на возможности ядра

Группа docker

Членство выдавайте как права администратора; альтернатива — rootless-режим

Rootless Docker

Демон работает без root, ущерб при побеге из контейнера заметно меньше

usersns-remap

Сопоставление идентификаторов: root внутри контейнера не root на узле

Профили seccomp и AppArmor

Ограничение системных вызовов; стандартный профиль лучше не отключать

Удалённый доступ к API

Не открывайте порт 2375 без TLS и проверки клиентских сертификатов — это открытый root

Журналы и квоты

Ограничивайте размер журналов: `--log-opt max-size=10m --log-opt max-file=3`

Compose: безопасный шаблон

```
services:
  api:
    image: registry.example.com/api@sha256:...
    user: "1000:1000"
    read_only: true
    tmpfs: ["/tmp"]
    cap_drop: ["ALL"]
    security_opt: ["no-new-privileges:true"]
    ports: ["127.0.0.1:8080:8080"]
    mem_limit: 512m
    pids_limit: 200
    restart: unless-stopped
    logging:
      options: { max-size: "10m", max-file: "3" }
```

Реестр и цепочка поставки

- Свой реестр с доступом по учётным данным; публичные образы — только из проверенных источников.
- Образы по отпечатку, а не по изменяемому тегу — так выкладка воспроизводима.
- Подпись артефактов и проверка подписи на стороне кластера.
- Перечень компонентов (SBOM) рядом с образом: позволяет быстро оценить новую уязвимость.
- Права сборочного конвейера ограничены: сборка не должна иметь доступа к рабочей среде.

Признаки неблагополучия

- Контейнер неожиданно грузит процессор — частый признак постороннего майнера.
- Исходящие соединения на незнакомые адреса; ограничьте исходящий трафик по умолчанию.
- Образы неизвестного происхождения в реестре или на узле.
- Порты, открытые наружу, хотя должны быть только локальными: сверяйте `docker ps` и `ss -ltnp`.
- Контейнеры, работающие месяцами без обновления базового образа.